COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	10/13/26	**DATE SUBMITTED:**	10/15/26
MEMBER/S:	Egar, Jarel Dave Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

Pre-Lab Tip:
When you are given a problem to solve using dynamic programming, the trick is "Not to over-think, because the solution is always hidden in the problem". Most beginners make this mistake of thinking too much, only because they thought the problem was difficult and only a complex solution would solve it. But in most cases the solution turns out to be simple.

Before you start coding, formulate the problem (not the solution) mathematically on a piece of paper. Because a problem well formulated is a problem half solved.

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

For the pre-lab review, you must fill out the table below. Using algorithms implemented in the previous activities, identify which algorithm satisfies the given algorithmic strategy. For the analysis, you must explicitly provide what part of the algorithm you have identified matches the corresponding strategy.

Strategy	Algorithm	Analysis
Recursion		
Brute Force		
Backtracking		
Greedy		
Divide-and-Conquer		

This table must be included in your section 6 as table 12-1.

The Problem: MINIMUM STEPS TO ONE

On a positive integer, you can perform any one of the following 3 steps.

1.) Subtract 1 from it. (n = n - 1)
2.) If its divisible by 2, divide by 2. (If n % 2 == 0, then n = n / 2)
3.) If its divisible by 3, divide by 3. (If n % 3 == 0, then n = n / 3).

Now the question is, given a positive integer n, find the minimum number of steps that takes n to 1.

Examples:

1.) For n = 1, output: 0
2.) For n = 4, output: 2 (4 /2 = 2 /2 = 1)
3.) For n = 7, output: 3 (7 -1 = 6 /3 = 2 /2 = 1)

One can think of greedily choosing the step, which makes n as low as possible and conitnue the same, till it reaches 1. If you observe carefully, the greedy strategy doesn't work here.

Eg: Given n = 10 , Greedy --> 10 /2 = 5 -1 = 4 /2 = 2 /2 = 1 (4 steps).

But the optimal way is --> 10 -1 = 9 /3 = 3 /3 = 1 (3 steps).

So, we need to try out all possible steps we can make for each possible value of n we encounter and choose the minimum of these possibilities.

It all starts with recursion.

$$F(n) = 1 + min\{F(n-1), F(n/2), F(n/3)\}\ if\ (n > 1), else\ 0\ (i.e., F(1) = 0).$$

Now that we have our recurrence equation, we can right way start coding the recursion. Wait. Does it have over-lapping subproblems? YES. Is the optimal solution to a given input depends on the optimal solution of its subproblems? Yes... Bingo! it is dynamic programming. So, we just store the solutions to the subproblems we solve and use them later, as in memorization, or we start from bottom and move up till the given n, as in dynamic programming. As it's the very first problem we are looking at here, let's see both the codes:

Memoization

```
int memo[n+1]; // we will initialize the elements to -1 ( -1 means, not solved it yet )
int getMinSteps ( int n )
{
    if ( n == 1 ) return 0; // base case
    if( memo[n] != -1 ) return memo[n]; // we have solved it already :)
    int r = 1 + getMinSteps( n - 1 ); // '-1' step . 'r' will contain the optimal answer finally
    if( n%2 == 0 ) r = min( r , 1 + getMinSteps( n / 2 ) ) ; // '/2' step
    if( n%3 == 0 ) r = min( r , 1 + getMinSteps( n / 3 ) ) ; // '/3' step
    memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.
    return r;
}
```

Run this code and show the output in table 12-2.

Bottom-Up DP

```
int getMinSteps ( int n )
{
    int dp[n+1], i;
    dp[1] = 0; // trivial case
    for( i = 2 ; i < = n ; i ++ )
    {
        dp[i] = 1 + dp[i-1];
        if(i%2==0) dp[i] = min( dp[i] , 1+ dp[i/2] );
        if(i%3==0) dp[i] = min( dp[i] , 1+ dp[i/3] );
    }
    return dp[n];
}
```

Run this code and show the output in table 12-3.

Both the approaches are fine. But one should also take care of the lot of over head involved in the function calls in Memoization, which may give StackOverFlow error or TLE rarely.

ILO A Output/s:

Pre-Lab Review:

Strategy	**Algorithm**	**Analysis**
Recursion	Merge Sort	Recursion is applied in this algorithm according to our previous lab activity right after the dataset is divided in half where **merge sort** is **recursively** called within the main function to process the **first** and **second halves**.
Brute Force	Linear Search	Brute forcing appears in the lab algorithm for linear search at about the second step where it **checks every single item** in the array.
Backtracking	Depth-First Search Traversal Techniques (Pre-, In-, & Post-order)	Backtracking happens in all of the Depth-First Search Traversal Techniques **implicitly** after the recursive calls are executed where each traversal

		may go back up the tree to process the next step.
Greedy	Selection Sort	In our selection sort algorithm, taken from the lab, the greedy strategy can be seen at **Routine_Smallest()** where it chooses the best option available at the moment of the unsorted sub array without consideration for anything else.
Divide-and-Conquer	Merge Sort or Binary search	Divide and conquer happens in phases within **merge sort** where '**divide**' happens when the array is split in half, and **conquer**' happening when the split halves are then recursively processed. For Binary Search, the strategy happens when '**no**' is compared with '**mid**' and cuts the search space in half.

Table 12-1. Algorithmic Strategies and Examples

<table>
<tr><td>Screenshot</td><td colspan="2">

Memoization.cpp > main()

```cpp
#include <iostream>
#include <algorithm>
#include <vector>

using namespace std;

// Maximum value for n (adjust if testing larger numbers)
const int MAX = 100000;
int memo[MAX]; // Global array, initialized in main

int getMinSteps(int n)
{
    if (n == 1) return 0; // Base case[cite: 2]

    if (memo[n] != -1) return memo[n]; // We have solved it already :)[cite: 2]

    int r = 1 + getMinSteps(n - 1); // '-1' step. 'r' will contain the optimal answer finally[cite: 2]

    if (n % 2 == 0) {
        r = min(r, 1 + getMinSteps(n / 2)); // '/2' step[cite: 2]
    }

    if (n % 3 == 0) {
        r = min(r, 1 + getMinSteps(n / 3)); // '/3' step[cite: 2]
    }

    memo[n] = r; // Save the result. If you forget this step, then it's same as plain recursion.[cite: 2]
    return r;
}

int main()
{
    int n = 10; // Change n to test different values

    // Initialize memo array with -1 (-1 means not solved yet)[cite: 2]
    for (int i = 0; i <= n; i++) {
        memo[i] = -1;
    }

    cout << "Minimum steps for n = " << n << " is: " << getMinSteps(n) << endl;

    return 0;
}
```

```
Minimum steps for n = 10 is: 3
```

</td></tr>
<tr><td rowspan="2">Analysis</td><td>Egar:</td><td>This code demonstrates how memoization (Top-Down approach) is properly used. The code shows how to get the minimum steps to get 1 for a given N. For example, N is 10, according to the problem the algo evaluates it by 10 - 1 = 9, 10/2 = 5. The code used recursion to find the best option out of the 3, with option 3 as the best at finding the minimum steps.</td></tr>
<tr><td>Medina:</td><td>The code above shows the implementation of top-down dynamic programming in order to find the minimum number of operations needed to lower a number down to 1 with the use of subtracting by 1 and dividing by either 2 or 3 if applicable. Additionally, recursion is applied to find the best choice for minimizing the total step count. Furthermore, caching is implemented within the code, allowing for an optimized sequence where already calculated values are stored temporarily and are just used instead of calculating the entire line again.</td></tr>
</table>

Table 12-2. Memoization Implementation

<table>
<tr><td>Screenshot</td><td colspan="2">

```cpp
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int getMinSteps(int n)
{
    // Use std::vector instead of fixed array to avoid 'constant value' error
    vector<int> dp(n + 1);

    dp[1] = 0; // Trivial case

    // Fixed lowercase 'for'
    for (int i = 2; i <= n; i++)
    {
        dp[i] = 1 + dp[i - 1];

        if (i % 2 == 0) {
            dp[i] = min(dp[i], 1 + dp[i / 2]);
        }

        if (i % 3 == 0) {
            dp[i] = min(dp[i], 1 + dp[i / 3]);
        }
    }

    return dp[n];
}

int main()
{
    int n = 10; // Change n to test different inputs (e.g., 10, 100)

    cout << "Minimum steps for n = " << n << " is: " << getMinSteps(n) << endl;

    return 0;
}
```

```
Minimum steps for n = 10 is: 3
```

</td></tr>
<tr><td rowspan="2">Analysis</td><td>Egar:</td><td>This code however uses a bottom-up dynamic programming method to find which option has the least steps taken to reach 1. In memoization (top-down) it uses recursion to find the best option, however in a bottom-up method it uses iteration (loops) instead of recursion.</td></tr>
<tr><td>Medina:</td><td>This time, the implemented code shows that of bottom-up dynamic programming in order to find the minimum number of steps needed just like earlier. From what I've observed, it seems to be using a vector (allowing for a dynamically manipulated array) and is sequentially computing the solutions. The algorithm evaluates the valid transitions (subtracting by 1 and dividing by either 2 or 3 just like with memoization) for each integer, 'i' followed by storing the minimum step count in dp[i], doing it on top of already solved states.</td></tr>
</table>

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. SUPPLEMENTARY OUTPUT

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

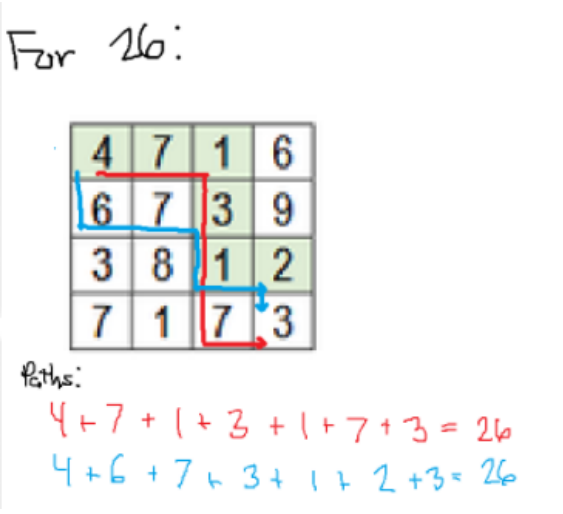
The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple "subproblems", which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:

- Pseudocode (+ Solving Problem by Hand)
- Working C++ Code
- Analysis of working code
- Screenshots of demonstration

SUPPLEMENTARY OUTPUTS:

Pseudocode (& Solving Problem by Hand):

Pseudocode:	Solving Problem by Hand
FUNCTION pathFind(row, column, remain) IF row or column is outside matrix return 0 ELSE subtract current cell cost from remain IF remain < 0 return 0 IF current cell is destination IF remain == 0 return 1 ELSE return 0 right = pathFind(row, column + 1, remaining) down = pathFind(row + 1, column, remaining) RETURN right + down FUNCTION END START Store matrix Print matrix User input of the required cost Print pathFind(0, 0, required cost) END	 The above shows the two possible paths solved by hand in consideration for a cost of 26.

Working C++ Code

main.cpp > main()

```cpp
#include <iostream>

const int ROWS = 4;
const int COLS = 4;

int matrix[ROWS][COLS] = {
    // Lab supple given matrix
    {4, 7, 1, 6},
    {6, 7, 3, 9},
    {3, 8, 1, 2},
    {7, 1, 7, 3}
};

int pathFind(int row, int column, int remain){
    if (row >= ROWS || column >= COLS) {
        return 0;
    }

    remain = remain - matrix[row][column];
    if (remain < 0){
        return 0;
    }

    if (row == ROWS - 1 && column == COLS - 1){
        if (remain == 0) {
            return 1;
        }
        return 0;
    }

    int right = pathFind(row, column + 1, remain);
    int down = pathFind(row + 1, column, remain);
    return right + down;
}
```

```cpp
int main() {
    int cost;

    std::cout << "4x4 Matrix Given:\n";
    for (int row = 0; row < ROWS; row++){
        for (int column = 0; column < COLS; column++){
            std::cout << matrix[row][column] << " ";
        }
        std::cout << "\n";
    }

    std::cout << "=======\nEnter path cost to find: ";
    std::cin >> cost;

    std::cout << "\nNumber of paths: " << pathFind(0, 0, cost) << std::endl;

    return 0;
}
```

Analysis:

Egar:	The code demonstrated the use of backtracking strategy by recursively locating all of the possible paths that the code will take through the matrix and avoiding the paths that do not meet the requirement or that are not in the designated grids that the code provided. It tests every right and down move step-by-step while subtracting each visited square's value from the main goal. If a path goes off the board or exceeds the total sum, it instantly stops that route and backtracks to try another direction.
Medina:	The code above follows exactly as the pseudocode created where it basically only has two possible movements (either right or down) and follows the simple following flow. Generally, it works by checking the position where if there is no valid path found, it **returns 0**. Additionally, it also goes over automatically subtracting the cell costs (including the middle), followed by checking the remaining cost where if the cost goes below 0 (negative) it stops exploring that specific path (because it goes over the cost). Furthermore, it also only ever **returns 1** or a **successful path** (where the cost is exactly zero) and finally has a way of adding valid paths together from both the **right** and **down** function calls in the case that there are multiple valid paths from where they are landing (like in the case that there are 2 valid paths for **right** and 1 for **left**, meaning there are 3 when they are added together).

Demonstration:

```
PS C:\Users\TIPQC\Downloads\t
4x4 Matrix Given:
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3
=======
Enter path cost to find: 26

Number of paths: 2
PS C:\Users\TIPQC\Downloads\t
4x4 Matrix Given:
4 7 1 6
6 7 3 9
3 8 1 2
7 1 7 3
=======
Enter path cost to find: 21

Number of paths: 1
```

Conclusion/s:

Egar:

In conclusion this HOA has helped us understand the application of algorithmic strategies like Recursion, Backtracking, Brute Force, Greedy, Backtracking and Divide and Conquer. ILO A provided me with an overview for what the topic is all about by making us analyze and find algos that match each strategy. ILO B created an example of the Memozation (top-down) and tabulation (bottom-up) method of the dynamic programming strategy and showed how they find the best option with the minimal steps. And lastly the supplementary activity used a recursive backtracking using a 4x4 matrix and slowly moving its way through the grid based on the given path to get the wanted value.

Medina:

In conclusion, this activity has taught me a lot about algorithmic analysis, the procedure directly overlapping with the lecture that was taken from the session accordingly. That is, going over the different algorithmic strategies such as recursion (using a function inside a function to reach a base case), brute force (trying everything), backtracking (exploring a path and going back), greedy (finding the most optimal locally but perhaps not globally), and divide-and-conquer (splitting a structure over and over and then processing them before merging them back). Moreover, the supplementary challenges us by passing as a matrix to handle on which we processed it to find the path with the given cost via recursion. Overall, I think I did well in this activity and only need some proper time to review and grasp all the concepts taught.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.